

Detection of SQL Antipatterns in jOOQ Database Access Code Using Large Language Models

Kristo Isberg

Supervisor: Erki Eessaar, PhD

Tallinn University of Technology

23.04.2026

Background

- **SQL Antipatterns**
 - "... a commonly occurring solution to a problem that generates decidedly negative consequences" (Brown et al., 1998)
 - Can affect performance, maintainability, portability, and data integrity
 - **Example:** *Implicit Columns* (using `SELECT *`) can waste >25% of runtime/energy
- **jOOQ:** Popular Java library for type-safe, DSL-based database access

SQL Antipatterns in Java: literature review

- Prevalent in Java projects
 - *Implicit Columns* affects every 50th query
 - Remain unfixed longer than traditional code smells
 - Authors suspect lack of awareness or low priority
- Two tools capable of statically detecting SQL antipatterns in Java
 - None capable of detecting from jOOQ DSL

Research Goals

- Development of an LLM-based static analysis tool for detecting SQL antipatterns
- Studying the prevalence and occurrence patterns of SQL antipatterns in jOOQ

Research Questions

- **RQ1:** How do different LLMs compare in identifying SQL antipatterns in Java code using jOOQ for database access?
- **RQ2:** How do different prompting strategies affect LLMs' accuracy in identifying SQL antipatterns in Java code using jOOQ for database access?
- **RQ3:** How accurate is the developed LLM-based tool in detecting SQL antipatterns in Java code using jOOQ for database access?
- **RQ4:** What are the occurrence patterns of SQL antipatterns in Java code using jOOQ for database access?
- **RQ5:** Which jOOQ API methods are most frequently associated with SQL antipattern occurrences?

Dataset Creation

- Used GitHub API to mine for projects using jOOQ
 - Filtered out non-Java projects, duplicates, etc
 - Gathered **602** applicable projects
- The author annotated a subset of 10% projects
 - Stratified sampling using head-tail breaks to preserve size distribution
 - 4 large, 10 medium, 47 small projects
 - **1,562** total antipattern occurrences identified manually

Dataset Creation

- Internal consistency measured using Cohen's Kappa
 - Re-annotated 10% of files after one month

$$\kappa = \frac{p_o - p_e}{1 - p_e} = 0.834$$

- Result: *Almost perfect agreement*
- **61** projects split into training/validation/test sets
 - Split ratio of 34/33/33
 - Monte Carlo optimisation to achieve an optimal split

Analysis of LLMs and prompting strategies

- Models:
 - OpenAI GPT-5.2 — reasoning model
 - Anthropic Claude 4.5 Opus — non-reasoning* model
 - Z.ai GLM-5 — open model
 - OpenAI gpt-oss-120B — medium-sized open model
- Prompting strategies:
 - Zero-Shot — baseline
 - Few-Shot — providing examples
 - Chain-of-Thought — "Think step by step."
 - Tree-of-Thought — simulating a conversation between three experts

Analysis of LLMs and prompting strategies

- Designed prompts for each prompting strategy, evaluating against training set
- Evaluated performance for each LLM and strategy combination on validation set
 - Precision, Recall, F1-Score
- Localisation task: True Positive only if model found the correct lines according to Intersection over Union

$$\text{IoU} = \frac{|S_p \cap S_g|}{|S_p \cup S_g|} \geq 0.5$$

RQ1: How do different LLMs compare in identifying SQL antipatterns in Java code using jOOQ for database access?

- All large models demonstrated similar performance
 - Varying costs and runtimes
 - GPT-5.2 was more paranoid than others
- gpt-oss-120B suffered from anomalies degrading performance

Model	Precision	Recall	F1-Score	Cost (USD)	Runtime (s)
GPT-5.2 (reasoning)	0.85	0.92	0.88	26.16	2,423
GLM-5 (reasoning)	0.88	0.89	0.88	11.11	5,775
Claude Opus 4.5 (non-reasoning)	0.88	0.89	0.88	29.97	367
gpt-oss-120B (reasoning)	0.83	0.87	0.83	1.49	2,808

RQ2: How do different prompting strategies affect LLMs' accuracy in identifying SQL antipatterns in Java code using jOOQ for database access?

- 2% performance gains to non-reasoning model from CoT and ToT
- Up to 1% gains to open models from Few-Shot
- Gains for one model generally balanced by degradations for other models
- Large increases to cost and runtime

Strategy	Avg F1-Score Increase	Avg Cost Increase	Avg Runtime Increase
Few-Shot	0.6%	31%	-1%
Chain-of-Thought	-0.3%	23%	90%
Tree-of-Thought	0.1%	88%	638%

Analysis tool development and evaluation

- **Stack:** Bun (build tool & runtime), TypeScript, Vercel AI SDK.
- **Architecture:** Self-contained CLI tool
- **Pre-processing:**
 - Irrelevant files skipped based on black- and whitelists
 - **Line Number Prefixing:** Crucial to help LLMs maintain spatial awareness and prevent miscalculations
- Zero-Shot prompt, default model Claude Opus 4.5 (non-reasoning)
- **Output:** Human-readable text summaries + machine-readable JSON/CSV.

RQ3: How accurate is the developed LLM-based tool in detecting SQL antipatterns in Java code using jOOQ for database access?

- Overall Precision: **0.95** | Recall: **0.91** | F1-Score: **0.92**
- **Best Performers:** *Implicit Columns* (0.97) and *ID Required* (0.94)
- **Worst Performers:** *Keyless Entry* (0.76) and *Fear of the Unknown* (0.82)
 - **Why?** Require cross-file context

RQ4: What are the occurrence patterns of SQL antipatterns in Java code using jOOQ for database access?

- 15,931 total antipatterns found in 602 projects, 26.5 per project
- **Dominant Core:** "Implicit Columns" and "ID Required"
 - Contained by 89% and 87% of projects
 - High project-wise Jaccard Index (0.81) and Spearman correlation (0.43)
- **Missing Constraints:** "Keyless Entry" and "Beware of the Unknown"
 - High project-wise Jaccard Index (0.34) and Spearman correlation (0.32)
- **Infection Path:** Rarer SQL antipatterns are strong indicators of more common ones
 - High project-wise (0.95) and file-wise (0.54) conditional probability

RQ5: Which jOOQ API methods are most frequently associated with SQL antipattern occurrences?

Implicit Columns

- More than 75% associated with shorthands for fetching jOOQ records
 - e.g. `DSL.selectFrom(TABLE)`, `DSL.select().from(TABLE)`
- Only 12.9% explicitly show the intent to fetch all columns
 - e.g. `DSL.asterisk()`, `TABLE.fields()`

RQ5: Which jOOQ API methods are most frequently associated with SQL antipattern occurrences?

Poor Man's Search Engine

- More than 60% explicitly use `LIKE` and `ILIKE` operators with wildcards
 - `FIELD.like("%" + value + "%")`, `FIELD.ilike`
- Wildcards are added implicitly in 28% of cases
 - `FIELD.contains(value)`, `FIELD.containsIgnoreCase`

Highlights & Contributions

- Created a high-quality ground truth
- First to treat LLM-based antipattern detection as a localisation task
- Developed the first available tool for detecting SQL antipatterns in jOOQ
- Performed a large-scale analysis of jOOQ projects

Thank you for listening!